# Paper Replication + Extension

## Candidate Papers

CSC/DSE 589 — Scientific Computing — Fall 2026

**Proposal due:** Fri Sep 18, start of class

**What this project is.** In place of the capstone, 589 students take one paper from this list, implement its central method from scratch, reproduce a result the paper reports, and then extend it — either by applying the method beyond the paper's scope or by benchmarking it against a modern alternative. The deliverable is a written report with working code.

**Your proposal sets your own schedule.** These projects are not interchangeable, and a single set of dates imposed across all of them would fit none of them. Your one-page proposal should say what you will have working by when, in your own terms. We will agree on it, and that becomes your schedule.

**You may propose a paper not on this list.** Bring it to office hours before Sep 18. The bar is that it connects to the course spine and that its central result is implementable in a semester.

### How to read this list

Every paper here is a genuine classic and every one is a good project. They differ enormously in *what kind* of work they are, and you should pick on that basis rather than on which sounds most impressive.

- ♦ marks a paper that requires you to build substantial supporting machinery *before* you can attempt the paper's actual contribution. These are the most systems-heavy projects on the list. They are not harder intellectually; they are larger.
- **Course coverage** tells you when we cover the background in lecture. Most of these land in the second half of the semester — so for most papers you will be reading ahead. That is normal for a graduate project, and it is worth knowing before you commit.

---

### 1. Limited-memory BFGS

Nocedal, J. (1980). "Updating Quasi-Newton Matrices with Limited Storage." *Mathematics of Computation* **35**(151), 773–782. [DOI]

**In plain terms.** Newton's method for optimization converges very fast, but it needs the matrix of second derivatives — $n^2$ numbers, which is hopeless once $n$ is large. This paper shows you can keep most of Newton's speed while storing only the last handful of steps, typically five to twenty vectors. It is the default optimizer in a great deal of scientific software, and the reason large-scale optimization is practical.

**What you would reproduce.** The two-loop recursion, and the paper's comparison of convergence against full BFGS and plain gradient descent as the memory parameter $m$ varies.

**A natural extension.** Apply it to a problem with gradients supplied by automatic differentiation rather than by hand; or investigate how $m$ should scale with problem conditioning.

**What it takes.** Moderate. The recursion is short; the fiddly part is a line search satisfying the Wolfe conditions, which you must implement and which is where most of the debugging time goes. *Course coverage: line search S37, Nov 18; BFGS and L-BFGS S40, Nov 30.*

## 2. ♦ Checkpointing for reverse-mode differentiation

Griewank, A. (1992). "Achieving Logarithmic Growth of Temporal and Spatial Complexity in Reverse Automatic Differentiation." *Optimization Methods and Software* **1**(1), 35–54. [DOI]

**In plain terms.** Automatic differentiation computes exact derivatives of a *program* — not a formula — by recording every arithmetic operation it performs and then replaying the recording backwards. This works beautifully until the recording gets too large to fit in memory, which happens almost immediately for anything containing a long loop: simulate a million timesteps and you must store a million intermediate states. This paper shows you can keep only a handful of snapshots and recompute the rest on demand, buying an enormous reduction in memory for a modest number of extra recomputations — and getting bit-for-bit the same derivative.

**What you would reproduce.** The central tradeoff result: with $c$ checkpoints and recomputation factor $r$, the number of steps you can reverse is $\binom{c+r}{c}$. Verify it empirically, then measure real memory and runtime and show both growing like $\log T$ rather than like $T$.

**A natural extension.** Apply it to the 489 capstone's implicit ODE integrator, or benchmark your hand-rolled schedule against `Checkpointing.jl` or `Enzyme.jl`.

**What it takes.** ♦ The largest project on this list. Reverse-mode AD is a prerequisite, not the subject — the paper assumes you already have it. **Working reverse-mode AD will be supplied to you as starter code** so that the project is the checkpointing schedule and the experiments, not the differentiation engine. Even so, expect real systems work: code that decides which snapshots to keep, machinery to save and restore program state, and honest memory measurement. *Course coverage: S32–S34, Nov 6–11 — later than you will need it, which is why the scaffold is provided.*

## 3. Randomized SVD

Halko, N., Martinsson, P.-G. & Tropp, J. A. (2011). "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions." *SIAM Review* **53**(2), 217–288. [arXiv:0909.4061]

**In plain terms.** The SVD tells you the best possible low-rank approximation to a matrix, but computing it exactly for a very large matrix is expensive. This paper shows something surprising: multiply your matrix by a few *random* vectors and you capture almost all of its important structure. You get a near-optimal answer far faster, together with a proof of how likely you are to be close. It is one of the most influential numerical papers of the last twenty years.

**What you would reproduce.** The core algorithm is about fifteen lines. Reproduce the paper's accuracy curves as you vary how many random vectors you use, and show what changes on matrices whose structure is less sharply defined — the case where the basic method struggles and the paper's

refinement of it earns its keep.

**A natural extension.** Apply it to a real dataset and compare against a truncated exact SVD on both accuracy and wall time; or test how the error bounds hold up on matrices whose spectra violate the paper's assumptions.

**What it takes.** The lightest implementation on this list — Julia's `LinearAlgebra` gives you `randn`, `qr`, and `svd`, and the algorithm is a short composition of them. The substance is in the experiments and in the probabilistic analysis, which is the most mathematically demanding of any paper here. *Course coverage: S19, Oct 7.*

## 4. GMRES

Saad, Y. & Schultz, M. H. (1986). "GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems." *SIAM Journal on Scientific and Statistical Computing* **7**(3), 856–869. [DOI]

**In plain terms.** Some linear systems are far too large to solve by elimination — millions of unknowns, with a matrix nobody could store as a plain grid of numbers. Iterative methods attack these instead: start from a guess and improve it repeatedly, using nothing but the ability to multiply the matrix by a vector. GMRES is the workhorse of that family for general matrices, including the badly behaved ones that more specialized methods cannot touch. At each step it has accumulated a small set of candidate directions, and it chooses the combination of them that leaves the smallest possible error. The catch is that both cost and storage grow with every step taken, which forces the "restart" compromise the paper introduces — and restarting can cause the method to stall completely.

**What you would reproduce.** The full algorithm — accumulating the candidate directions, and picking the best combination of them at each step — together with the restarted variant that caps memory use, and the paper's convergence comparisons. Demonstrate the stagnation that restarting can cause.

**A natural extension.** Add preconditioning and quantify the improvement; or compare against BiCGSTAB on the same problems.

**What it takes.** Moderate, and entirely self-contained — roughly eighty lines of careful bookkeeping with no external dependencies. *Course coverage: none. GMRES is deliberately outside the 489 syllabus, so you would be the only person in the room who knows it. Krylov subspace ideas are S24–S26, Oct 19–23.*

## 5. Adam — and the proof that was wrong

Kingma, D. P. & Ba, J. (2015). "Adam: A Method for Stochastic Optimization." *ICLR 2015*. [arXiv:1412.6980]

Pair with: Reddi, S. J., Kale, S. & Kumar, S. (2018). "On the Convergence of Adam and Beyond." *ICLR 2018* (best paper). [arXiv]

**In plain terms.** Adam is the optimizer that trains most neural networks. It adjusts the step size separately for every parameter, using running averages of recent gradients. It is one of the most cited papers in machine learning — and its convergence proof is wrong. A later paper constructed a simple problem on which Adam provably fails to converge, and won a best-paper award for it. This is the one project on the list where the point is not to confirm a result but to break one.

**What you would reproduce.** The update rule is about ten lines. Reproduce the paper's benchmark comparisons, then implement Reddi et al.'s counterexample and show Adam failing on it.

**A natural extension.** Implement AMSGrad and verify it fixes the counterexample; or examine whether the failure mode appears in realistic problems or only in the adversarial construction.

**What it takes.** Trivial to implement, which means the entire project is experimental design and careful argument. Choose this one only if you want to write a critical paper rather than a systems one — a report that just says "Adam works" is not a replication. *Course coverage: S39, Nov 23.*

## 6. ♦ Nested dissection

George, A. (1973). "Nested Dissection of a Regular Finite Element Mesh." *SIAM Journal on Numerical Analysis* **10**(2), 345–363. [DOI]

**In plain terms.** When you factor a sparse matrix, zeros turn into nonzeros — "fill-in" — and a matrix that fit comfortably in memory suddenly does not. How much fill you suffer depends entirely on the *order* in which you eliminate the unknowns, and choosing well is worth orders of magnitude. This paper gives a recursive ordering for grid problems — cut the grid in half, handle each half, then handle the seam — and proves it is optimal in order of magnitude. Every serious sparse direct solver still uses a descendant of this idea.

**What you would reproduce.** The ordering algorithm on a regular grid, and the paper's two central claims — $O(N \log N)$ fill and $O(N^{3/2})$ operations — against the natural ordering's $O(N^{3/2})$ and $O(N^2)$.

**A natural extension.** Extend from regular grids to irregular meshes, where separators stop being geometric and a real partitioner becomes necessary; or compare against a modern partitioner such as METIS.

**What it takes.** ♦ A large project, but less exotic than it sounds. The ordering itself is short — perhaps thirty lines — because the paper works on a regular grid, where the seam you cut along is simply a row or column of grid points. Finding it takes no special machinery.

The hard part is measuring your result honestly. You need to count how much fill an ordering produces, and the obvious way to do that — factor the matrix, then count the nonzeros in the answer — quietly overcounts, by an amount that itself changes with problem size. The numbers look completely reasonable and the growth rate you conclude from them is wrong. **A correct counter is supplied as starter code**, along with the test matrices; you write the ordering and the experiments. *Course coverage: fill-in and reordering S23, Oct 16 — nested dissection is mentioned there but not developed.*

---

**Two honest warnings.** The ♦ papers are larger, not deeper. If you want the most mathematically demanding project on this list, that is Halko et al., which is also the easiest to code. And if you pick Adam, the burden is on you to produce a critical result — reproducing a well-known method's headline numbers is not, by itself, a replication.